

Muhammad Saad Khan (251309919)

1)

MODIFIED-COUNTING-SORT(A, k)

1. let $C[0..k]$ be a new array
2. for $i = 0$ to k
3. $C[i] = 0$
4. for $j = 1$ to $A.length$
5. $C[A[j]] = C[A[j]] + 1$
6. // $C[i]$ now contains the count of elements equal to i
7. $j = 1$
8. for $i = 0$ to k
9. while $C[i] > 0$
10. $A[j] = i$
11. $j = j + 1$
12. $C[i] = C[i] - 1$

2)

Insert 35



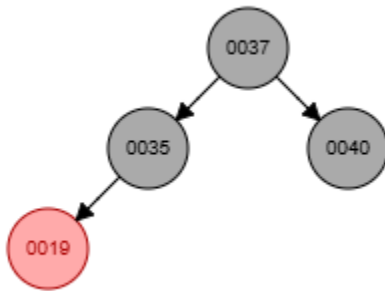
Insert 40



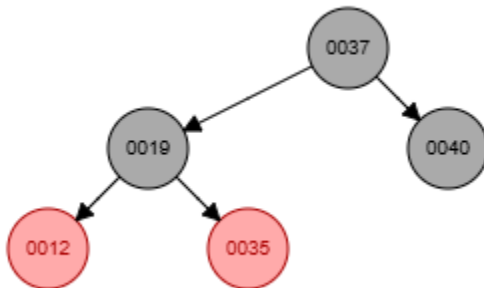
Insert 37



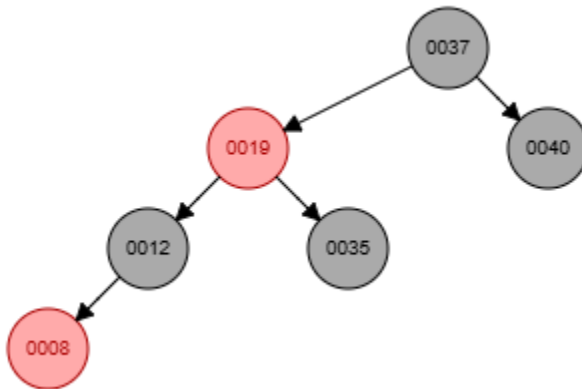
Insert 19



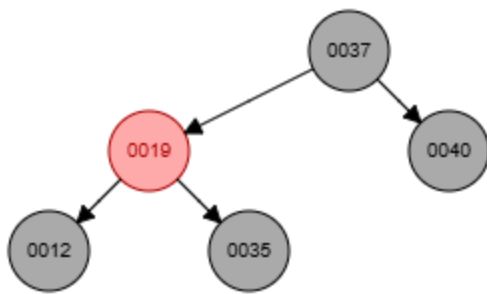
Insert 12



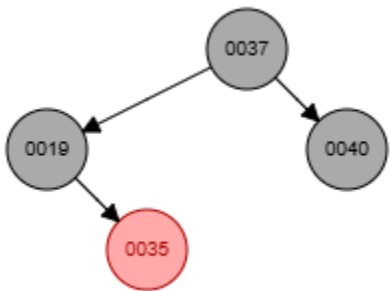
Insert 8



3)
Delete 8



Delete 12



Delete 19



Delete 37



Delete 40



Delete 35
No nodes

4)

Base Case:

After zero operations, the heap is empty. After the first insert there is one node with rank 0, and $\lfloor \lg 1 \rfloor = 0$.

Assume after the first t operations, every node in a heap of size n_t satisfies $\text{rank} \leq \lfloor \lg n_t \rfloor$. Consider the $(t+1)$ -th operation.

Insert: A new node with rank 0 is added. Since $\lfloor \lg(n_t+1) \rfloor \geq 0$, the bound holds for the new node. Existing nodes are unaffected.

Union: Merges two root lists. No ranks change. The combined heap has $n' \leq n_t$ nodes (no new nodes created).

Delete: Removes one node then consolidates, during consolidation nodes are linked when two roots of equal rank k are linked, one becomes the child of the other, increasing that parent's rank to $k+1$. The total number of nodes decreases by 1 so $n' = n_t - 1$

5)

$\lceil \lg \lg n \rceil$ bits are necessary to store $x.\text{rank}$ for each node x

Yes 1 byte = 8 bits which can represent the range of 0 to 255. If $\lceil \lg \lg n \rceil$ bits are necessary to store $x.\text{rank}$ for each node x means the maximum is $\lceil \lg n \rceil$ so $\lceil \lg n \rceil > 255 \rightarrow n > 2^{255}$.

Algorithms:

Final set:

```
int final_sets() {
    finalized = true;
    std::map<int,int> remap; int counter = 1;
    for (int i = 1; i <= total_elements; i++) {
        if (parent[i] == 0) continue;    // skip non-members
        int r = find_set(i);
        if (remap.find(r) == remap.end())
            remap[r] = counter++;    // assign new label
    }
    new_label.assign(total_elements+1, 0);
    for (int i = 1; i <= total_elements; i++) {
        if (parent[i] == 0) continue;
        new_label[i] = remap[find_set(i)];    // store normalized label
    }
    return num_sets;
}
```

Correctness: every element i that was make_set receives a label in $\{1, \dots, \text{num_sets}\}$ such that two elements share a label if and only if they are in the same set.

Complexity: $O(n \log n)$

Algorithm for 6b):

CONNECTED-COMPONENTS(image, rows, cols)

1. $n \leftarrow \text{rows} \times \text{cols}$
2. $\text{ds} \leftarrow \text{new DisjointSet}(n)$
3. $\text{index}(r, c) \leftarrow r \times \text{cols} + c + 1$
4. for $r \leftarrow 0$ to $\text{rows} - 1$
5. for $c \leftarrow 0$ to $\text{cols} - 1$
6. if $\text{image}[r][c] == '+'$
7. $\text{ds.make_set}(\text{index}(r, c))$
8. for $r \leftarrow 0$ to $\text{rows} - 1$
9. for $c \leftarrow 0$ to $\text{cols} - 1$
10. if $\text{image}[r][c] == '+'$
11. if $\text{image}[r-1][c] == '+'$
12. $\text{ds.union_sets}(\text{index}(r, c), \text{index}(r-1, c))$

```
13.         if image[r][c-1] == '+'
14.             ds.union_sets(index(r,c), index(r,c-1))

15. K ← ds.final_sets()

16. for r ← 0 to rows - 1
17.     for c ← 0 to cols - 1
18.         if image[r][c] == '+'
19.             label[r][c] ← ds.label(index(r, c))
20.             size[label[r][c]] ← size[label[r][c]] + 1

21. sort components by size descending
22. assign letter 'a' to largest, 'b' to second, ..., up to 'z'

23. print original image
24. print label grid (all components)
25. print sorted list of (letter, size)
26. print label grid (size > 1 only)
27. print label grid (size > 4 only)
```